

Aula 11

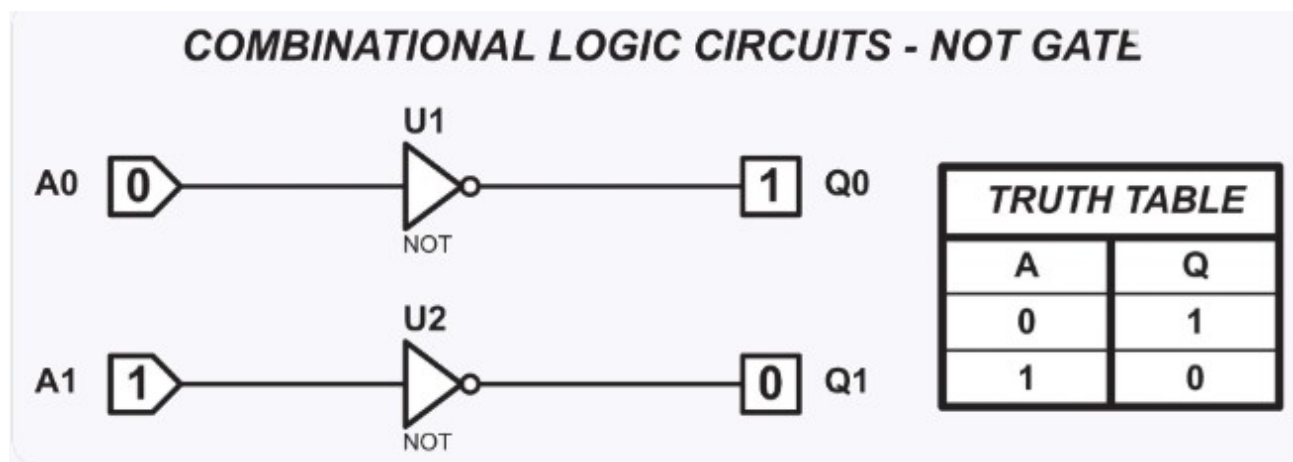
1. Introdução à Álgebra de Boole

Diferente da álgebra elementar que lida com números reais, a álgebra booleana opera sobre um conjunto de apenas dois elementos: $\{0,1\}$, frequentemente interpretados como Falso (0) e Verdadeiro (1).

O Operador Binário

Um operador binário é uma regra que combina dois elementos de um conjunto para produzir um terceiro elemento do mesmo conjunto. Na lógica, os três operadores fundamentais são:

1. Soma Lógica (OR/OU): Representada por $+$. Resulta em 1 se ao menos uma das entradas for 1.
2. Produto Lógico (AND/E): Representado por $-$. Resulta em 1 apenas se ambas as entradas forem 1.
3. Complemento (NOT/NÃO): Um operador unário representado por A' ou $\bar{A}$, que inverte o valor da entrada.



2. Sistemas Algébricos

Um sistema algébrico booleano é definido por um conjunto B , dois operadores binários ($+$, $*$), um operador unário ($'$) e dois elementos distintos (0 e 1). Para que um sistema seja considerado uma Álgebra de Boole, ele deve satisfazer os **Postulados de Huntington**:

- Fechamento: Se $a, b \in B$, então $a+b \in B$ e $a \cdot b \in B$.
- Identidade: Existe um elemento neutro para cada operação.
 - $a + 0 = a$
 - $a \cdot 1 = a$
- Comutatividade: A ordem não altera o resultado.
 - $a + b = b + a$
 - $a \cdot b = b \cdot a$
- Distributividade: Uma operação se distribui sobre a outra.
 - $a \cdot (b + c) = (a \cdot b) + (a \cdot c)$
 - $a + (b \cdot c) = (a + b) \cdot (a + c)$. Nota: esta segunda forma é diferente da álgebra comum!
- Complemento: Para cada a , existe um a' tal que:
 - $a + a' = 1$
 - $a \cdot a' = 0$

3. Propriedades das Operações

Além dos postulados, derivamos teoremas fundamentais que auxiliam na simplificação de expressões lógicas:

Propriedade	Expressão (Soma)	Expressão (Produto)
Idempotência	$A + A = A$	$A \cdot A = A$
Absorção	$A + (A \cdot B) = A$	$A \cdot (A + B) = A$
Elemento Nulo	$A + 1 = 1$	$A \cdot 0 = 0$
Involução	$(A')' = A$	-
De Morgan	$(A + B)' = A' \cdot B'$	$(A \cdot B)' = A' + B'$

4. Funções Booleanas

Uma Função Booleana é uma expressão formada por variáveis binárias e operadores lógicos. Ela mapeia uma ou mais entradas binárias para uma única saída binária.

Representação

Uma função $f(x, y, z)$ pode ser representada de três formas principais:

1. Expressão Algébrica: Ex: $f = x * y + z'$
2. Tabela Verdade: Lista todas as combinações possíveis de entrada e o resultado da função.
3. Diagrama Lógico: Representação visual usando portas lógicas.

Formas Canônicas

Qualquer função booleana pode ser expressa de duas formas padronizadas:

- Soma de Produtos (SOP / **Mintermos**): Onde a função é representada pela união (OR) de termos que resultam em 1.
- Produto de Somas (POS / **Maxtermos**): Onde a função é representada pela interseção (AND) de termos que resultam em 0.

Exemplo Prático de Simplificação

Considere a função: $f = A * B + A * B'$

1. Aplicamos a distributividade: $A * (B + B')$
2. Pelo postulado do complemento, sabemos que $B + B' = 1$.
3. Logo: $A * 1$
4. Pelo elemento neutro: $f = A$.

No design de hardware, simplificar uma função booleana usando as propriedades acima significa reduzir o número de transistores necessários em um chip, economizando energia e espaço. Esta simplificação reduz custos em projetos de hardware, utilizando menos transistores para executar a mesma tarefa lógica.

Exemplo computacional

Para ilustrar a validação de um CPF (Cadastro de Pessoas Físicas) sob a ótica da computação e da lógica, utilizamos o Algoritmo do Módulo 11. Este é um exemplo clássico de como funções matemáticas e booleanas garantem a integridade dos dados.

1. A Lógica dos Dígitos Verificadores

Um CPF tem 11 dígitos no formato ABC . DEF . GHI - JK.

- Os 9 primeiros dígitos são o corpo principal.
- Os 2 últimos (J e K) são os dígitos verificadores, gerados a partir de cálculos lógicos sobre os 9 anteriores.

O Fluxo Lógico da Validação

A função de validação retorna um valor booleano (True para válido, False para inválido) seguindo estes passos:

1. Eliminação de Casos Óbvios: Se todos os dígitos forem iguais (ex: 111.111.111-11), a função retorna False imediatamente, embora o cálculo matemático possa bater.
2. Cálculo do Primeiro Dígito (J): Multiplica-se os 9 primeiros dígitos por pesos decrescentes de 10 a 2.
3. Cálculo do Segundo Dígito (K): Multiplica-se os 10 primeiros dígitos (incluindo o J recém-calculado) por pesos de 11 a 2.

2. Exemplo Prático de Cálculo

Vamos validar os dígitos para o início 123.456.789:

Passo 1: Encontrar o dígito J

Multiplicamos cada dígito pela sua posição (peso):

$$1(10) + 2(9) + 3(8) + 4(7) + 5(6) + 6(5) + 7(4) + 8(3) + 9(2) = 210$$

- Divide-se o resultado por 11: $210 / 11 = 19$ com **resto 1**.
- **Regra Booleana:** Se o resto é menor que 2, o dígito é 0. Caso contrário, o dígito é $11 - \text{resto}$.
- Como o resto é 1, **J = 0**.

Passo 2: Encontrar o dígito K

Agora incluimos o 0 (J) no cálculo:

$$1(11) + 2(10) + 3(9) + 4(8) + 5(7) + 6(6) + 7(5) + 8(4) + 9(3) + 0(2) = 255$$

- Divide-se 255 por 11: 23 com **resto 2**.
- Como o resto não é menor que 2, $11 - 2 = 9$. Logo, **K = 9**.

Resultado: O CPF válido para essa sequência seria 123.456.789-09.

3. Implementação Funcional (Python)

Aqui está como essa lógica se traduz em código funcional, aplicando os conceitos de iteração e condicionais booleanas:

```
def validar_cpf(cpf: str) -> bool:
    # Limpa formatação e verifica se tem 11 dígitos
    cpf = [int(digito) for digito in cpf if digito.isdigit()]
    if len(cpf) != 11 or len(set(cpf)) == 1:
        return False

    # Validação do primeiro dígito (J)
    soma_j = sum(cpf[i] * (10 - i) for i in range(9))
    resto_j = soma_j % 11
    esperado_j = 0 if resto_j < 2 else 11 - resto_j

    # Validação do segundo dígito (K)
    soma_k = sum(cpf[i] * (11 - i) for i in range(10))
    resto_k = soma_k % 11
    esperado_k = 0 if resto_k < 2 else 11 - resto_k

    # Retorna o resultado da expressão lógica
    return cpf[9] == esperado_j and cpf[10] == esperado_k

# Teste
print(validar_cpf("123.456.789-09")) # True
```

Por que isso é importante em programas?

1. Redução de Erros: Evita que dados "sujos" entrem no banco de dados.
2. Economia de Processamento: Validar no cliente (frontend) ou no início da requisição (backend) usando lógica simples evita consultas desnecessárias ao banco de dados ou APIs externas.
3. Segurança: É a primeira camada de defesa em sistemas de cadastro de usuários.